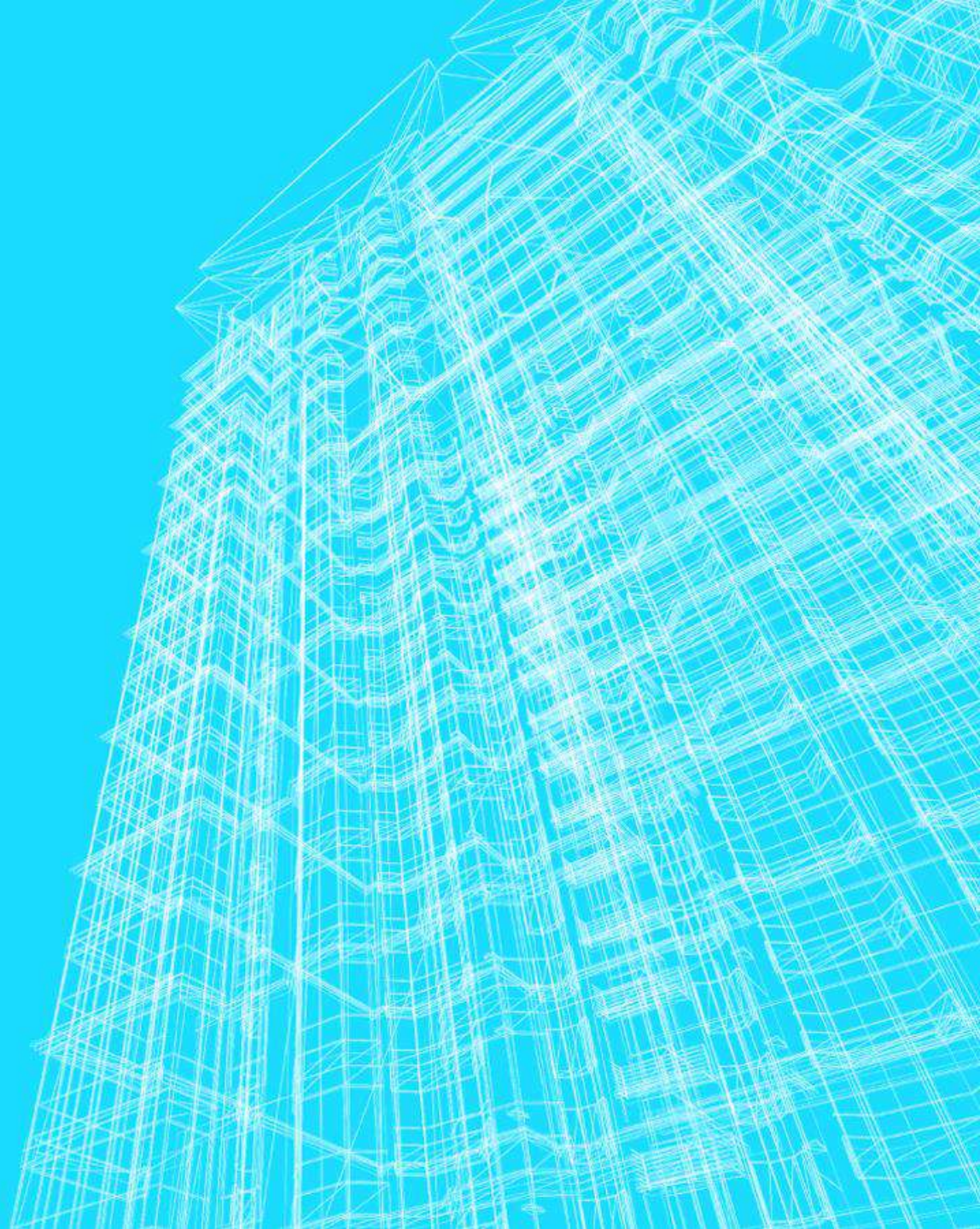


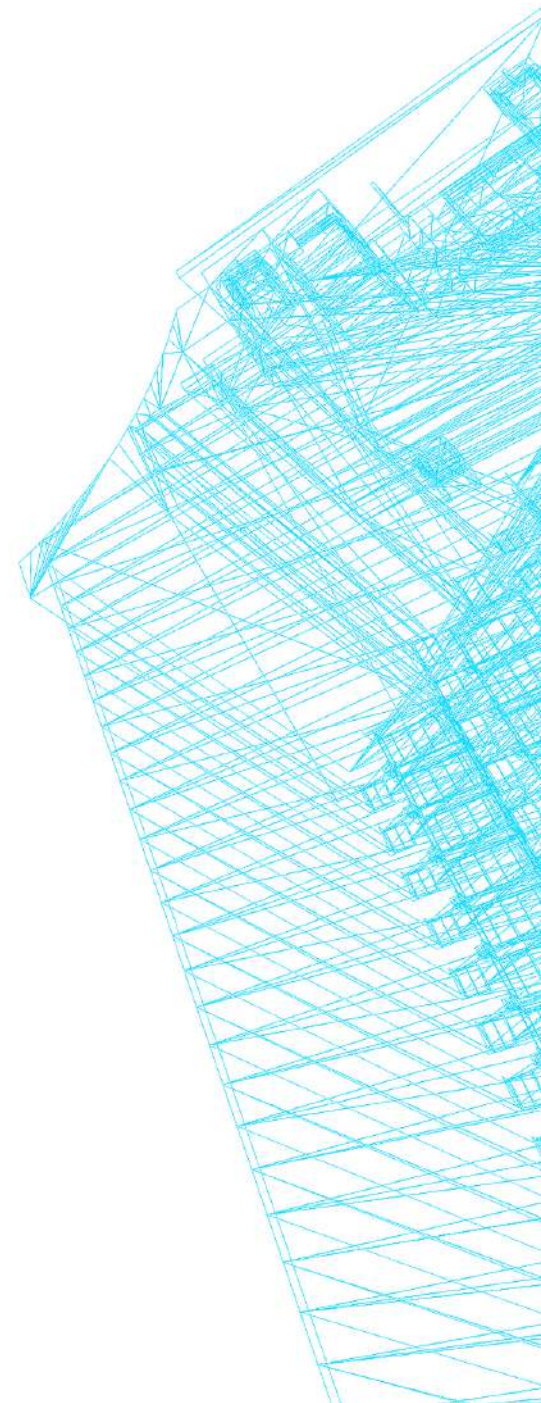
FORÇA BRUTA E TRANSFORMAÇÕES

João Pedro Oliveira Batisteli



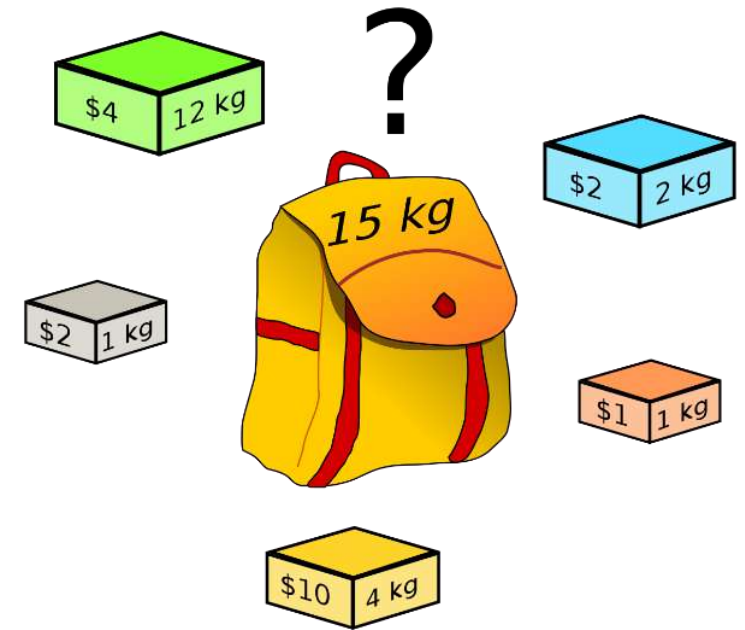
O PROBLEMA DA MOCHILA

Problemas Intratáveis



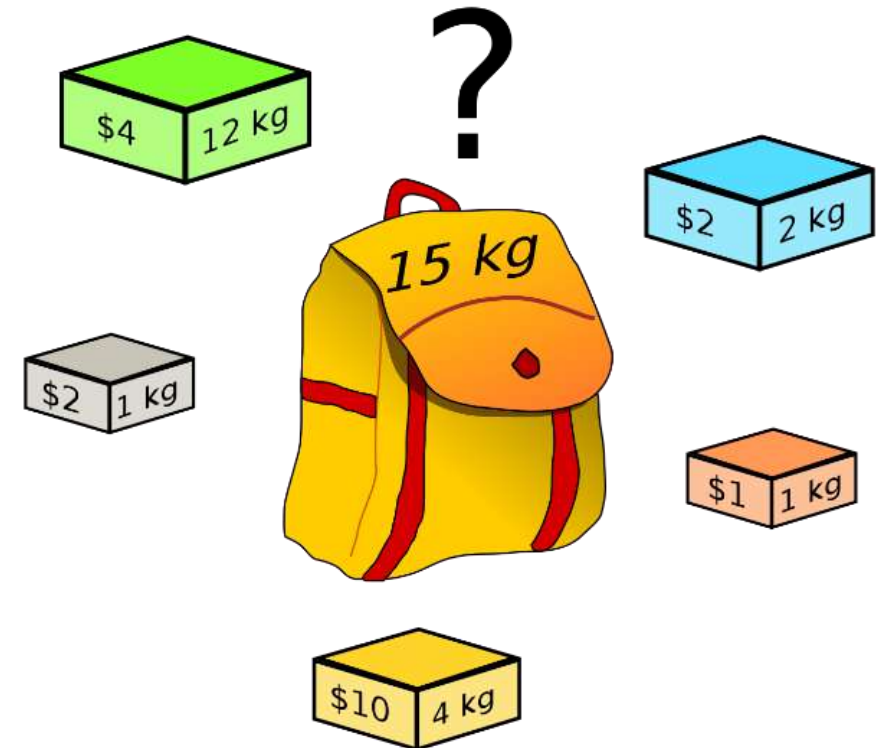
O PROBLEMA DA MOCHILA (KNAPSACK PROBLEM)

- Dados n itens, cada um com:
 - Pesos: $p_1, p_2, \dots, p_n$;
 - Valores: $v_1, v_2, \dots, v_n$;
- E uma mochila de capacidade C .
- Problema:
 - Encontrar o **subconjunto mais valioso** de itens que caibam dentro da mochila.



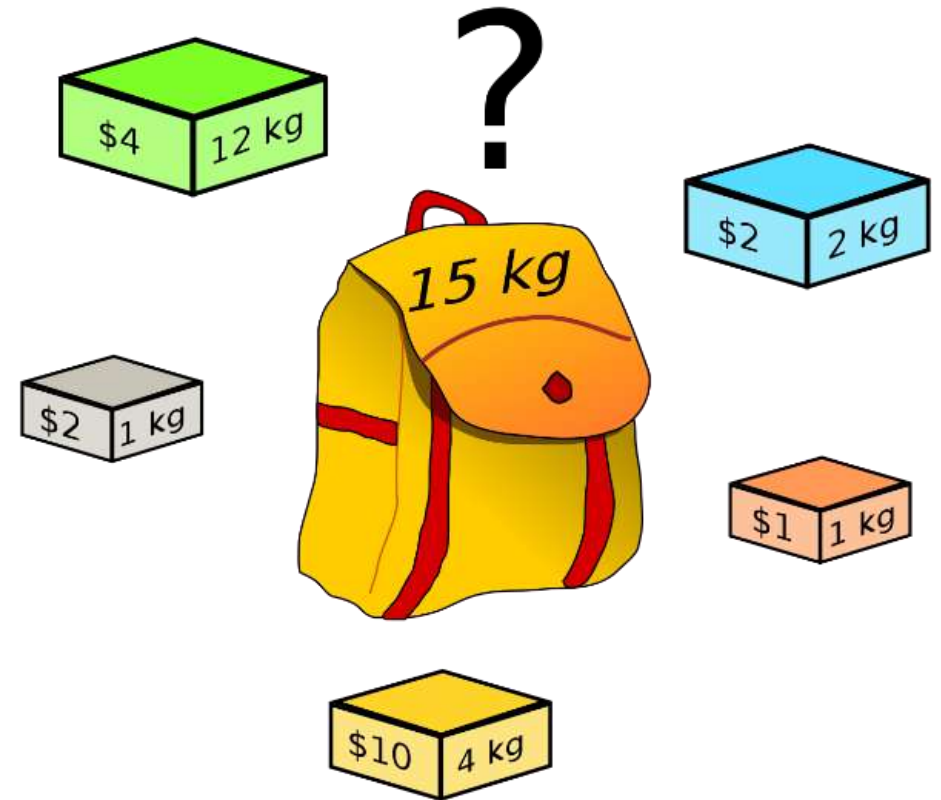
O PROBLEMA DA MOCHILA (*KNAPSACK PROBLEM*)

- Generalização para diversos problemas combinatórios:
 - Valor;
 - Restrição.

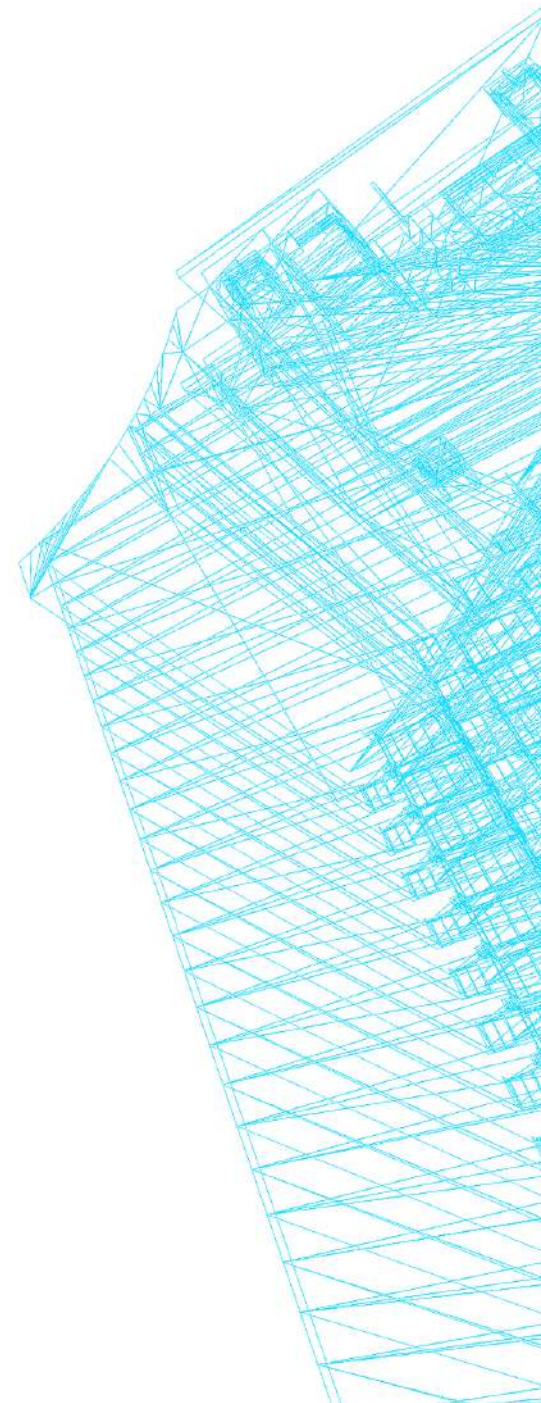


A MOCHILA

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2



ALGORITMOS DE FORÇA BRUTA



ALGORITMOS DE FORÇA BRUTA

- Travessia completa no espaço de busca.
 - Travessia *parcial* em melhores casos de execução.
- Abordagem direta do problema;
 - muitas vezes intuitiva (fácil de entender).
 - nem sempre fácil de implementar.
 - grande esforço computacional.

ALGORITMOS DE FORÇA BRUTA

- **Exploração Exaustiva:** Avalia todas as possibilidades dentro do espaço de busca.
- **Interrupção Precoce (Early Exit):** Possibilidade de travessia parcial nos melhores casos de execução.
- **Natureza Intuitiva:** Tradução direta da definição do problema para o algoritmo.
- **Geração do Espaço de Busca:** A lógica é simples, mas modelar e implementar combinações ou permutações pode ser trabalhoso.
- **Custo Computacional Elevado:** Geralmente resulta em complexidades de tempo inviáveis para grandes entradas, como exponenciais ($O(2^n)$) ou fatoriais ($O(n!)$).

FORÇA BRUTA

- **Eficiente** em problemas polinomiais.
 - Fatorial de um número, busca isolada em um conjunto, cálculo de somas e médias ...
- **Intratável** em problemas importantes exponenciais.
 - Ordenação de dados, caminho mínimo, alocação de recursos...

A MOCHILA COM FORÇA BRUTA

- A mochila com força bruta
 1. Listar todas as soluções potenciais para o problema;
 2. Avaliar as soluções, uma a uma;
 3. Encontrando a solução, retornar;

A MOCHILA COM FORÇA BRUTA

- A mochila com força bruta
 1. Listar todas as soluções potenciais para o problema;
 2. Avaliar as soluções, uma a uma;
 3. Encontrando a solução, retornar;
- 1. Gerar todos os subconjuntos de itens;*
- 2. Em cada subconjunto, avaliar peso e valor;*
 - 1. Armazenar o maior com peso válido;*
- 3. Retornar o subconjunto armazenado.*

A MOCHILA COM FORÇA BRUTA

- Gerar todos os subconjuntos de itens:

1

2

3

4

...

n

A MOCHILA COM FORÇA BRUTA

- Gerar todos os subconjuntos de itens:

1 - $\{1, 2\}, \{1, 3\}, \{1, 4\}, \dots, \{1, n\}$

2 - $\{2, 3\}, \{2, 4\}, \dots, \{2, n\}$

3 - $\{3, 4\}, \dots, \{3, n\}$

4 - $\{4, 4\}, \dots, \{4, n\}$

...

n

A MOCHILA COM FORÇA BRUTA

- Gerar todos os subconjuntos de itens:

1 - $\{1, 2\}, \{1, 3\}, \{1, 4\}, \dots, \{1, n\} - \{1, 2, 3\}, \{1, 2, 4\}, \dots, \{1, 2, n\}, \dots$

2 - $\{2, 3\}, \{2, 4\}, \dots, \{2, n\} - \{2, 3, 4\}, \dots, \{2, 3, n\}, \{2, 4, 5\}, \dots, \{2, 4, n\}$

3 - $\{3, 4\}, \dots, \{3, n\} - \{3, 4, 5\}, \dots, \{3, 4, n\}, \dots$

4 - $\{4, 4\}, \dots, \{4, n\} - \{4, 5, 6\}, \dots, \{4, 5, n\}, \dots$

...

n - $\{n\}$

MOCHILA: FORÇA BRUTA

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

Subconjunto	Peso Total	Valor Total
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

MOCHILA: FORÇA BRUTA

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

Subconjunto	Peso Total	Valor Total
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

MOCHILA: FORÇA BRUTA

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

Subconjunto	Peso Total	Valor Total
1,2	14	6
1,3	13	5
1,4	16	14
1,5	13	6
2,3	3	3
2,4	6	12
2,5	3	4
3,4	5	11
3,5	2	3
4,5	5	12

MOCHILA: FORÇA BRUTA

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

Subconjunto	Peso Total	Valor Total
1,2	14	6
1,3	13	5
1,4	16	14
1,5	13	6
2,3	3	3
2,4	6	12
2,5	3	4
3,4	5	11
3,5	2	3
4,5	5	12

MOCHILA: FORÇA BRUTA

Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

Subconjunto	Peso Total	Valor Total
1,2,3	15	7
1,2,4	18	16
1,2,5	15	8
1,3,4	17	15
1,3,5	14	7
1,4,5	17	16
2,3,4	7	13
2,3,5	4	5
2,4,5	7	14
3,4,5	6	13

MOCHILA: FORÇA BRUTA

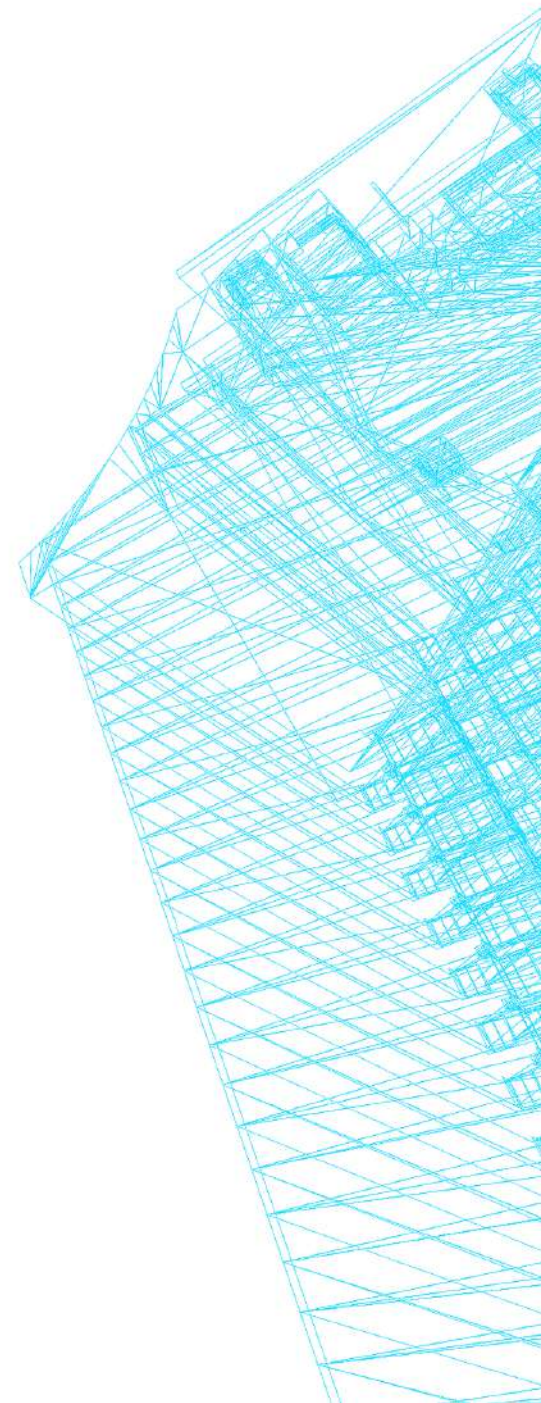
Item	Peso	Valor
1	12	4
2	2	2
3	1	1
4	4	10
5	1	2

Subconjunto	Peso Total	Valor Total
1,2,3	15	7
1,2,4	18	16
1,2,5	15	8
1,3,4	17	15
1,3,5	14	7
1,4,5	17	16
2,3,4	7	13
2,3,5	4	5
2,4,5	7	14
3,4,5	6	13

A MOCHILA COM FORÇA BRUTA

- Formas iterativa e recursiva.
- Útil para:
 - desenvolvimento rápido de algoritmos;
 - entrada **pequena**;
 - ou algoritmos que serão **executados poucas vezes**.

FORÇA BRUTA E TRANSFORMAÇÕES



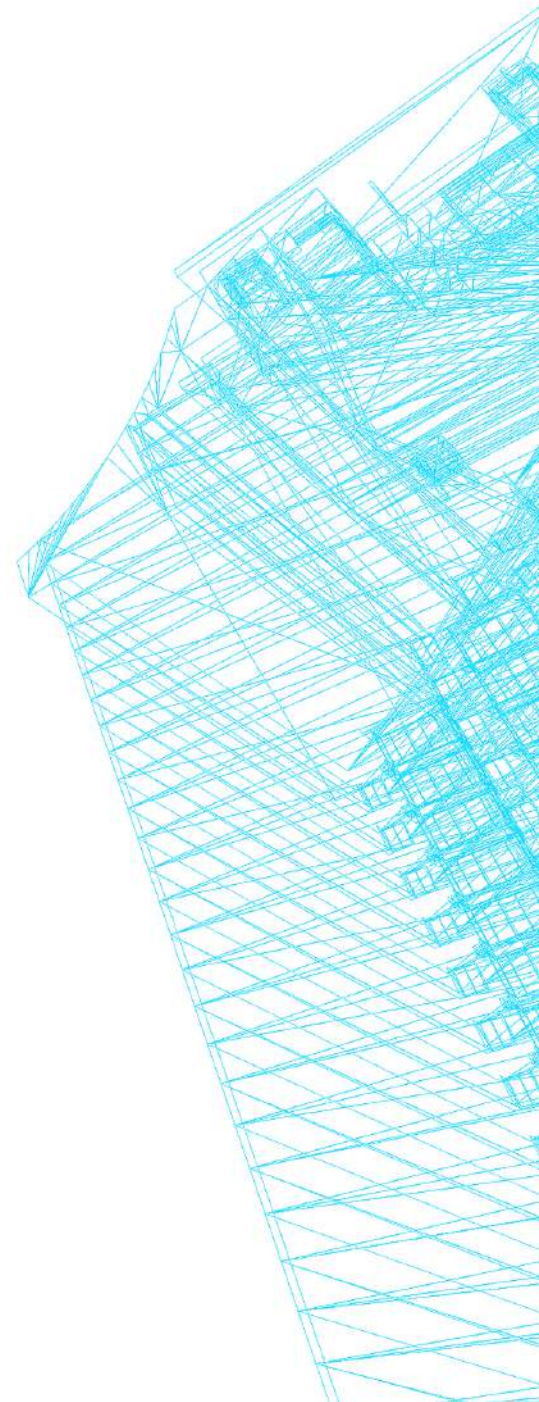
TRANSFORMAÇÃO

- Pode ser usada para o projeto de algoritmos recursivos ou não recursivos.
- *Transformar a instância do problema* visando facilitar a encontrar uma solução.
- Em seguida, a instância transformada é resolvida.

TRANSFORMAÇÃO

- Variações:
 1. Simplificação;
 2. Mudança de representação;
 3. Redução.

SIMPLIFICAÇÃO



TRANSFORMAÇÃO - SIMPLIFICAÇÃO

- Transformar para uma instância mais simples ou mais conveniente do problema.
 - **Ex:** existência de elementos repetidos em *array*.

TRANSFORMAÇÃO - SIMPLIFICAÇÃO

Ex: existência de elementos repetidos em *array*.

9	12	23	4	16	26	15	77	32	98	4	98
---	----	----	---	----	----	----	----	----	----	---	----

- Algoritmo de força bruta?
 - Complexidade?

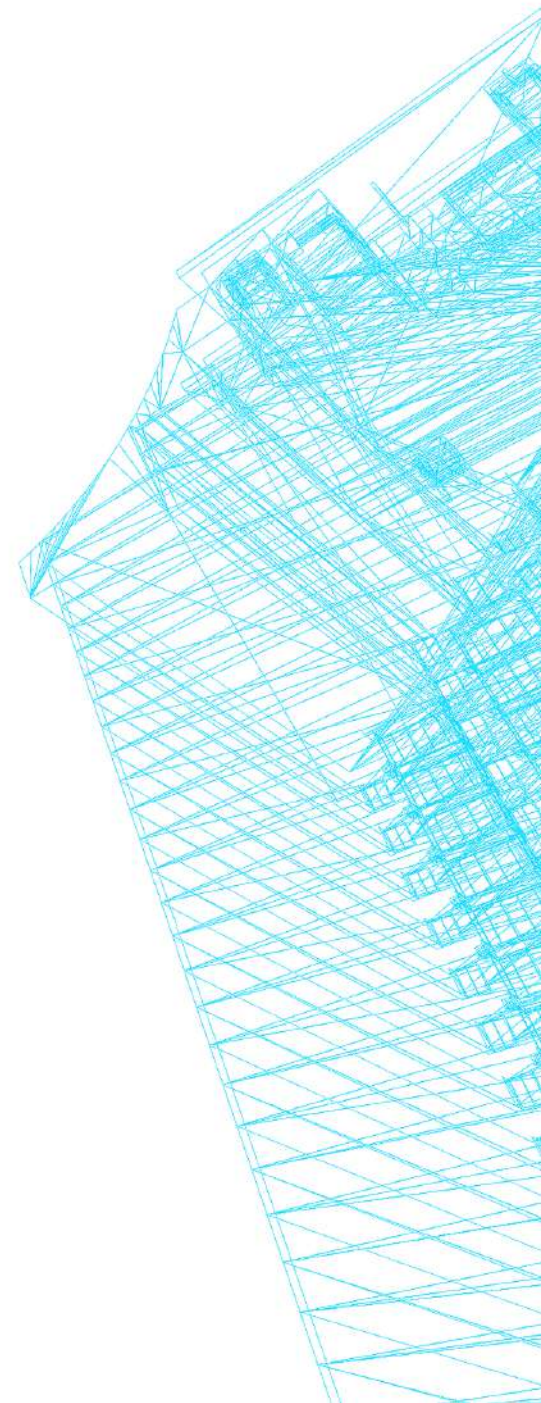
TRANSFORMAÇÃO - SIMPLIFICAÇÃO

Ex: existência de elementos repetidos em *array*.

4	4	9	12	15	16	23	26	32	77	98	98
---	---	---	----	----	----	----	----	----	----	----	----

- **Transformação:** ordenar os dados.
- Algoritmo?
- Complexidade?

MUDANÇA DE REPRESENTAÇÃO



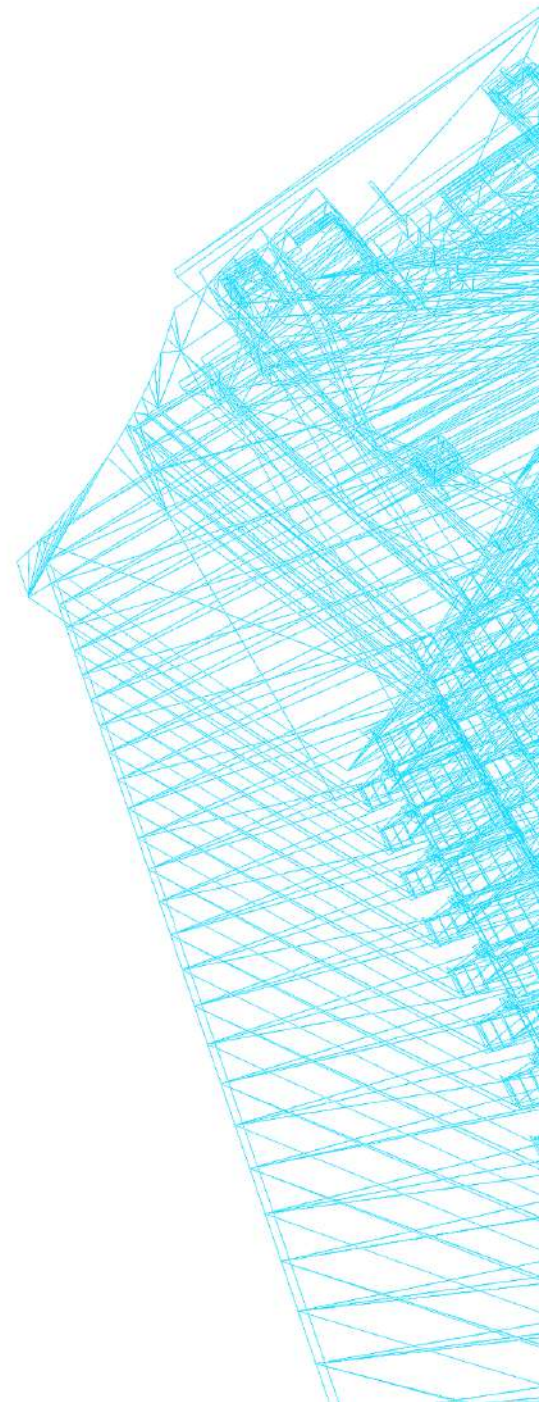
TRANSFORMAÇÃO - SIMPLIFICAÇÃO

- Transformação/transposição dos dados para uma **representação diferente**, na qual o problema é mais **facilmente resolvido**.
- **Ex:** existência de elementos repetidos em *array*.

TRANSFORMAÇÃO - SIMPLIFICAÇÃO

- Transformação/transposição dos dados para uma **representação diferente**, na qual o problema é mais **facilmente resolvido**.
- **Ex:** existência de elementos repetidos em *array*.
- **Inserir os elementos em uma tabela hash.**
 - **Complexidade?**

REDUÇÃO DO ESPAÇO DE BUSCA



REDUÇÃO DO ESPAÇO DE BUSCA

- Técnica indutiva (**decremental** ou **incremental**).
- Estratégia simples:
 - **Reduzir** a instância para uma instância menor;
 - **Resolver** a instância menor;
 - Caso necessário, **estender** a instância menor para obter a solução para o problema original.

PADRÕES EM STRINGS

- Sejam
 - Uma string s (tamanho n); um padrão p (tamanho m)
 - O padrão p (uma substring) ocorre dentro de s ?

PADRÕES EM STRINGS

- Sejam
 - Uma string s (tamanho n); um padrão p (tamanho m)
- Algoritmo
 1. Listar todas as soluções potenciais para o problema;
 2. Avaliar as soluções, uma a uma;
 3. Encontrando a solução, retornar;

PADRÕES EM STRINGS

- Sejam
 - Uma string s (tamanho n); um padrão p (tamanho m)
- Algoritmo
 1. Iniciar a conferência do padrão em cada posição de s ;
 2. Verificar todas as letras de p sucessivamente;
Chegando ao final de p , achamos o padrão
 3. Retornar a posição da conferência;

PADRÕES EM STRINGS

Uma string s (tamanho n); um padrão p (tamanho m)

```
para(i=0; i<(n-m+1); i++){
    j=0;
    enquanto( (j<m) e (p[j] == s[i+j]) ){
        j++;
    }
    se (j==m){
        retorne i;
    }
}
retorne -1;
```